

Assignment 1: Basics - Empirical Machine Learning

CS 312 · Ungraded

Instructions. This assignment is ungraded, but doing the assignment will significantly assist you in obtaining partial credit on your quiz and tutorial. Submit a single PDF writeup to Gradescope by the deadline in the course schedule.

For each problem, your deliverable will be two parts: a collection of experiments that you’ve run as part of your investigations into the phenomenon that we describe in the handout, and a short (around one paragraph) synthesis of your interpretation of the experiments. Your description of the experiment should be sufficiently well described (in some combination of hyperparameter changes, code diffs, and so on) that there is little room for interpretation on what you ran. Remember that *you will be able to cite your assignment in your quiz to justify your answer* and so your writeup must make it straightforward for us to check that whatever experiment you reference in your quiz actually exists, and does what you claim it does.

In the final problem, you can develop an interesting outcome prediction problem akin to our quizzes, where you will submit a short code diff from our codebase (under 100 lines) and a prediction task, framed as either multiple choice or a numerical prediction task. If we choose to use your submitted quiz question in any way (including in some modified form) on the actual quiz, you will receive bonus points equivalent to 1% of the total grade, and another 1% more if the question is hard (which we might define roughly as chance-corrected cross entropy $-\log p_{\text{students correct}} / \log K \geq 0.5$ for k-way multiple choice). Please clearly label your problem in gradescope if you want to be eligible for the bonus.

Overview *Empirical machine learning and our model organism*

In this assignment, you will gain the core skills for empirical machine learning (running experiments efficiently, drawing robust conclusions, tuning and debugging models) and become an expert on the small-scale language model that we will use as a “model organism”. Throughout this course, we will study deep learning by making small changes to this basic model, and so it is important that we have a good intuition for the behavior of this baseline.

We will organize this assignment around 4 sections with 7 core problems and a final optional problem, corresponding to questions that you will often find yourself asking as you do empirical machine learning and deep learning research.

- When do hyperparameters matter? Which ones?
- Will it scale?
- When is method A better than B?
- What’s going on with my model, and why isn’t it training?

These are fundamental questions and mastering even these questions will make you a much more effective researcher. We think that the best way to learn these intuitions and skills is through immersion: running hundreds of experiments and internalizing the results, and so expect each section to take 2-4 hours of active human time, and in total there should be around 170 model training runs at d8 or smaller scales. To help you with your time allocation, we have put in a suggested “number of experiments” in each experiment subsection.

Our model. Core to our class is a deliberately small but standard language model ([course repository](#)). It is small enough that you can run many experiments, but realistic enough that the failures, tradeoffs, and scaling behavior resemble larger language models.

Logging. The codebase naturally logs to weights and biases, and the codebase has hooks to store a number of useful statistics (like gradient norms) for your runs. Later in the assignment, you will be asked to look at and understand these statistics, so you should make sure your runs properly log to w&b and that you don't delete your statistics.

Objective. The model is trained for next-token prediction. Given a tokenized sequence $x_1, \dots, x_{1024}$, it predicts each next token x_{t+1} from the prefix $x_{\leq t}$. The training loss is the average cross-entropy over all predicted positions in the batch, and validation loss is the same quantity on held-out sequences. The default run uses 600,000 training sequences, or 614.4M training tokens.

Optimization. The standard recipe uses AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, peak learning rate 0.003, 1% linear warmup, and linear decay to zero. The batch size is 64 sequences, so each optimizer step sees 65,536 tokens. Weight decay is 0.1, with embeddings, normalization parameters, and biases excluded from decay, and gradients are clipped to norm 1.0. Dropout is off by default. A default run trains in roughly 10–15 minutes on a single H100.

Architecture. The default model is d8, a Llama-style decoder-only Transformer with 8 pre-norm blocks, hidden size 512, 8 attention heads, rotary position embeddings, RMSNorm, query-key normalization, and a SwiGLU MLP. It uses a 4096-token vocabulary, context length 1024, untied input/output embeddings, and has about 35M parameters. The built-in model family ranges from d4 through d18 by changing depth; width, MLP size, head layout, vocabulary size, context length, and related details are fixed for this ladder but can be changed with a custom model configuration.

What to take away from this assignment

Our model organism d8 is a very specific type of deep learning model, both in its setting (language modeling) and scale ($< 35M$ parameters). Lessons from d8 are *very* far from universal, and we will often find situations where phenomena on d8 will disagree with published results. Because of this, we must be very careful with what we take away from d8!

We ask you to master d8, sometimes even memorizing phenomena and losses, because we think strong mastery of a model organism is useful even though the phenomena itself won't transfer.

There are two concrete things we think that you can and should take away from these assignments and the class 1). for your own domain, you should seek to master the phenomena there as you did in this class, applying the same thought process and techniques and 2). you can use your intuition from this class as a very rough prior when you have no other source of information.

Running Experiments Well

Empirical machine learning is all about experimentation and experiment design. How can we spend the limited resources we have (in implementing experiments, and the compute in running them) and obtain reliable conclusions?

We will work on some of these skills for experiment design and develop better judgment for how to downscale experiments (and knowing when not to trust downscaled experiments). There are many good ways to do experiment design, but we will start by guiding you through an opinionated set of principles from the teaching staff.

- Change one thing at a time - try to decompose the hypothesis you have into a set of simpler tests.

- Use the minimal scale needed - scale down as much as you can to improve the number of experiments you can run.
- Know when things are real - estimate run-to-run variation to be able to identify robust vs ephemeral effects.
- Interpolate when you can - when possible, try to smoothly interpolate between your settings, rather than having a binary comparison.
- Use the right “batch size” - find the right number of parallel experiments to run at once, dependent on your compute environment.

AI tool use

You should feel free to use any AI assistance of your choice. The assignment is ungraded, and doing well on the quizzes requires you to fully understand your experiments and results. The final problem encourages you to come up with something novel and interesting that might inspire the teaching staff into making a new quiz question. For this reason, for the last question, it's in your best interest to try something different than the default that the AI comes up with.

Starter code and instructions

Clone the course repository and install its pinned dependencies with `uv sync`. Configure your username and other execution environment details in `utils.py`. The standard experiment code for each problem is documented in `experiments/a1_basics/p*_*.py`; the entrypoint is `launch_training_jobs(RUNS)`.

Hyperparameters

Hyperparameters, such as the learning rate or weight decay, have a major impact on the performance of models. The sweeps below illustrate how tuning can change the apparent benefit of an intervention. But hyperparameters are also very tricky to deal with - by their very nature, they are the parameters that we have not yet been able to reliably automate, and some human judgment is often needed to know when (and which) hyperparameters need to be tuned.

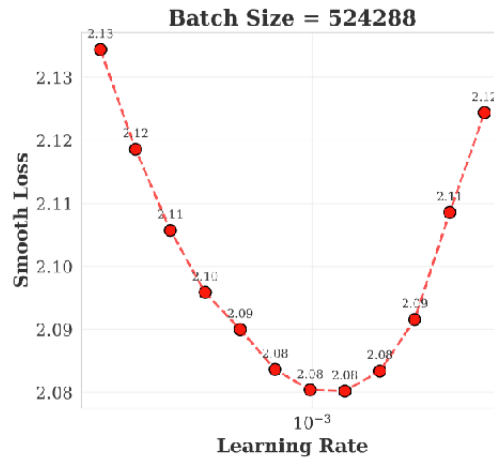
Since we are starting by studying language modeling, there are a few “basic” hyperparameters. We give a broad list here, categorized by hyperparameter type.

Hyperparameter	Default	Notes
Optimizer		
learning_rate	0.003	peak learning rate
optimizer_name	adamw	adamw or sgd
beta1	0.9	Adam β_1 (momentum)
beta2	0.95	Adam β_2
lr_schedule	linear	linear, cos, constant, $\text{wsd} \times \frac{\text{step}}{\text{total_steps}}$ (e.g. $\text{wsd}0.2$: stable then linear decay over the last 20%)
warmup_percent	0.01	linear warmup, as a fraction of total steps
weight_decay	0.1	masked: no decay on embeddings, norms, or biases
grad_norm	1.0	gradient-clipping threshold; set to None to disable
Data		
num_train_sequences	600,000	training horizon; each sequence is 1024 tokens
num_epochs	1.0	fractional values allowed, > 1 repeats data
batch_size	64	sequences per optimizer step
context length	1024	fixed by the preprocessed dataset
data_seed	42	shuffle order of training sequences
Model		
depth (d4–d18)	d8	built-in ladder: layers = depth, hidden = $64 \cdot \text{depth}$, heads = hidden/64, MLP = $3.5 \times \text{hidden}$, head_dim=64, vocab_size=4096, context length 1024. Custom LMConfigs can override these choices.
qk_norm	true	normalize queries and keys before attention dot products
tie_word_embeddings	false	share input and output embedding weights when true
model_seed	42	initialization randomness
dropout	0.0	shared attention and residual-branch dropout probability
Systems		
num_micro_batches	1	gradient accumulation chunks per optimizer step; usually only needed above batch size 256
deterministic	false	request deterministic PyTorch behavior when possible
precision	mp	mp, bf16, or fp32

A key task is *hyperparameter tuning*. Picking the best possible choice of hyperparameters. This is often quite complicated, due to the high-dimensionality of the hyperparameter space, and the subtle ways in which optimal hyperparameters can change by setting. Many papers have been invalidated due to subtle errors in baseline hyperparameter choice. Don’t let that be you!

There are a few key ideas to keep in mind. First is that some parameters are critical to re-tune and others are often set as rough defaults. Second, when we re-tune a hyperparameter, we want to test a

“reasonable range” of hyperparameters that have a good chance of containing an optimum. In many case, this involves a log-space grid, i.e. testing a base parameter $\text{base_param} * b^i$ for b an arbitrary base and i an integer-valued search range. You can see a visual example of this in a recent StepFun paper here,



Hyperparameter loss landscape shape from Houyi et al (Step Law) shows a nice, convex landscape when the learning rate is placed in log units.

Finally, many hyperparameters co-vary with each other, and you should not tune just one or the other, but instead tune the two together. Let us now try some practical examples to see what we mean by these ideas.

Problem 1: Warmup - hyperparameter sweeps and optimization (50 or so runs)

To start, let us do some basic hyperparameter optimization. Your goal is to take the standard *d8* training recipe and build a good mental model for how hyperparameters affect model performance (and how optimal hyperparameters co-vary). Throughout this question, we will be compute-matched, so you must keep the $\text{num_params} \times \text{num_total_tokens}$ fixed.

- Start by varying a few basic hyperparameters: warmup, learning rate, batch size, and weight decay, each on their own. Many of these parameters naturally live on a log-scale, and so you will want to take the default *d8* settings, a base multiplier (e.g. 3), and explore the shape of the loss as you vary the hyperparameter along this log axis (you could do 5 steps on each variable). After this investigation, you should have a good sense of what valid ranges are, what loss landscapes look like in hyperparameter space, which hyperparameters matter the most, and how you’d do a grid search. (15 runs)
- Some hyperparameters, like batch size and learning rate co-vary. Changing one can often mean that you need to also change the other in a controlled way. Identify the major co-varying hyperparameters among the set of (warmup, batch size, learning rate, weight decay) by picking pairs of hyperparameters that you think might be linked, and exploring the losses when you vary both hyperparameters. By the end, you should have a good mental list of hyperparameters that need to be tuned together, and how. (20 runs)
- Now explore how all of those components might now co-vary with the other optimizer hyperparameters, especially the learning rate scheduler. Try to get an understanding of how different schedulers generally interact with the above hyperparameters, and identify any key additional hyperparameters (15-20 runs)

In this first assignment, we will provide an example quiz question in each section to help contextualize the type of prediction tasks you will be expected to be able to do. Quiz questions will come in 3 flavors: easy, medium, and hard, where easy will require very little generalization from the assignment (if you literally run what we ask you to, and memorize the run results, you should be able to get the quiz right) while a hard question will be much more out of distribution, and we only expect students who do some exploration to reliably get partial credit via citing their assignments.

Here is an example of an “easy” question that tests your findings in part b.

Example question

Difficulty: Easy

We are interested in finding better hyperparameters for our d8 configuration. In following this goal, we are going to try increasing the batch size by a factor of 2, while tweaking our LR to match. We have 3 possible options for this experiment.

```
--- a/experiments/al_basics/p1_hyperparameters.py
+++ b/experiments/al_basics/p1_hyperparameters.py
@@
- RUNS = [
-     TrainConfig(),
- ]
+ RUNS = [
+     TrainConfig(
+         learning_rate=learning_rate,
+         batch_size=batch_size,
+     )
+     for learning_rate in [0.001, 0.003, 0.009]
+     for batch_size in [64, 128]
+ ]
```

Consider the following *three* configurations within the grid.

- A learning_rate = 0.003, batch_size = 64
- B learning_rate = 0.001, batch_size = 128
- C learning_rate = 0.009, batch_size = 128

Rank the three configurations from lowest to highest terminal loss, writing one letter in each box.

Prediction:

	<		<	
----------------------	---	----------------------	---	----------------------

Here is an example of a harder question, which more significantly tests generalization of your understanding of hyperparameters.

Example question**Difficulty: Medium**

Starting from the standard *d8* recipe, increase the peak learning rate to 0.009 and weight decay to 1.0. Keep all other settings standard, including the 600,000-sequence training budget.

```
--- a/experiments/al_basics/p1_hyperparameters.py
+++ b/experiments/al_basics/p1_hyperparameters.py
@@
+RUNS = [
+    TrainConfig(learning_rate=0.009, weight_decay=1.0,
+                lr_schedule=s, warmup_percent=w)
+    for s, w in [("wsd0.2", 0.0), ("wsd0.2", 0.2), ("linear", 0.1)]
+]
```

Rank A–C from lowest (best) to highest (worst) final validation loss.

Run	lr_schedule	warmup_percent
A	wsd0.2	0.0
B	wsd0.2	0.2
C	linear	0.1

Prediction:

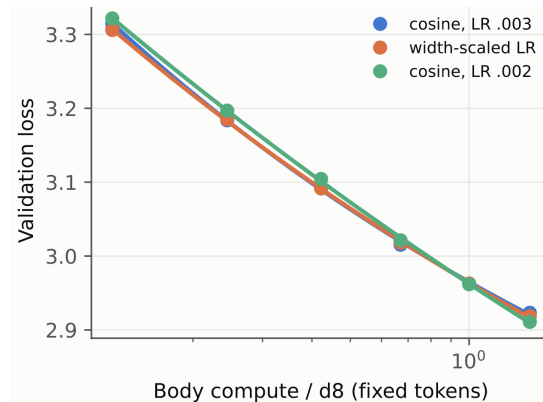
	<		<	
----------------------	---	----------------------	---	----------------------

Scaling

Predictable returns to compute means that scale (in our model size and data quantity) is a particularly important parameter to choose when running experiments. Understanding the right notion of scale, and how your model scales is critical to effective empirical machine learning.

Background

In many cases, if we pick our axes correctly, then we see a highly regular trend in how performance scales with compute,



Recorded d4–d9 pilot fits (left) show reasonable scaling

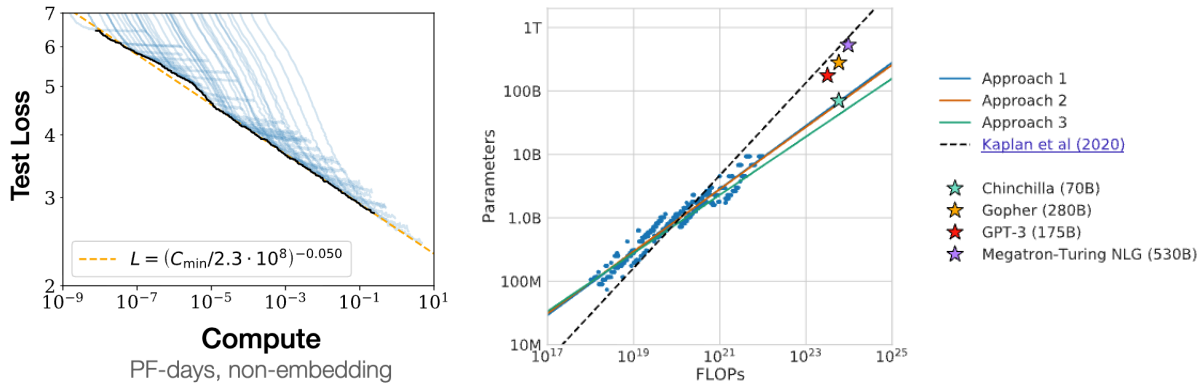
In many cases, this is well-modeled via a power law ($L = (\frac{1}{C})^\alpha + \epsilon$ for the scaling axis C , slope α and irreducible error ϵ), though in general we would expect this to be a sigmoidal shape interpolating from random guessing / initialization to the irreducible error as scale goes to infinity.

Under this type of parametric fit, we can predict the behavior of models at compute scales far larger than our training runs. We can think of the testing problem as making these forecasts given a fixed scaling recipe (i.e. how we change the model as we vary the x-axis) and the engineering problem as how we define the scaling recipe itself.

It will turn out that both of these problems are hard, and we can't take scaling for granted. There are many cases of scaling trends misleading us, especially if we are not careful with how we design our scaling recipe.

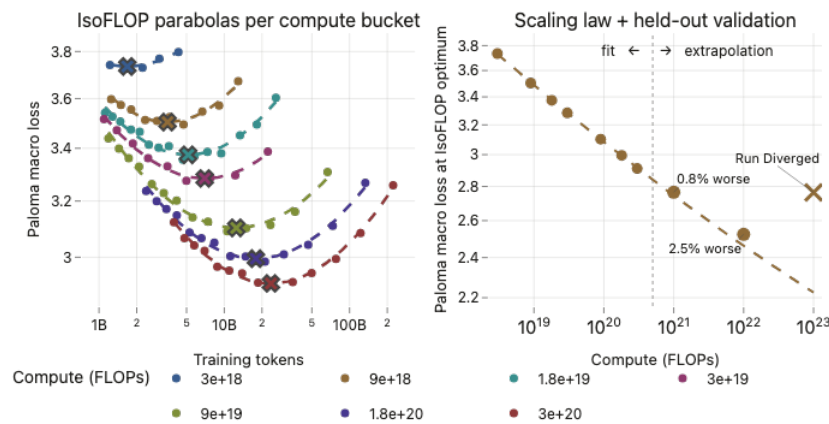
Even when our scaling law fits look good, extrapolation across many orders of magnitude can be quite tricky, and it turns out that scaling law accuracy is both a technically challenging and economically extremely important problem. Here are some examples of how this can be tricky.

In “Scaling Laws for Neural Language Models”, Kaplan et al find a high accuracy scaling law (see left plot) for how model performance depends on FLOPs and model size. However, later on, in “Training Compute-Optimal Large Language Models”, Hoffmann et al find subtle issues with these scaling extrapolations and show that much smaller models are compute optimal relative to Kaplan et al's estimates.



Scaling law disagreement between Kaplan et al's original scaling laws, and the "Chinchilla" laws from Hoffmann et al.

For a more controlled, recent example, the Marin project trains open language models. In a recent blog post (<https://openathena.ai/blog/delphi/>), the marin researchers discuss their failed scaling extrapolation, in which they find that even near perfect fits up to a certain point can begin to rapidly diverge due to training instabilities.



Failed scaling law from the Marin project. Scaling law looks near perfect up to 10^{21} but blows up soon after.

Forecasting scale dependent performance

Let us begin by building intuition for 3 different cases that might happen when you are comparing two methods' scaling trends.

1. **Linear, fixed slope:** The two methods share a slope, so gains are scale independent (in log space).
2. **Linear, varying slope:** The two methods vary with scale, but do so in a predictable, shared linear space, making estimation of the crossover point possible.
3. **Nonlinear:** The two methods may seem linear, but there exist nonlinear deviations which make multiple order of magnitude predictions difficult.

Problem 2: Predicting the reliability of a scaling law (50 runs)

In this question, we will fix the scaling recipe, training models between d4 and d9 scale, letting the width vary together with depth (all integer depths, including odd ones like d5 or d9, are available

in the codebase; see the hyperparameter table above).

- (a) As a starting point, build 4 scaling law plots across the d4 to d9 scale, testing the following variations of the d8 model 1). baseline model with other hyperparams set to d8 2). constant LR schedule, 3). linear LR schedule, dropout = 0.2 4). linear LR schedule, learning rate=0.03. Before training d8 and d9, pre-register your scaling predictions, and then run d8 and d9. What is your best prediction for scales far out, all the way to d20? (20 runs)

```
experiments/a1_basics/p2_scaling_law_reliability.py
RUNS = []
seen_configs = set()
for depth in range(4, 10):
    for learning_rate, lr_schedule, dropout in [
        (0.003, "linear", 0.0),
        (0.003, "constant", 0.0),
        (0.003, "linear", 0.2),
        (0.03, "linear", 0.0),
    ]:
        config_key = (depth, learning_rate, lr_schedule, dropout)
        if config_key in seen_configs:
            continue
        seen_configs.add(config_key)
        RUNS.append(
            TrainConfig(
                model_config=depth_model_config(depth),
                learning_rate=learning_rate,
                lr_schedule=lr_schedule,
                dropout=dropout,
            )
        )
```

- (b) We now want to find interventions which alter the slope without destroying a useful scaling trend. Explore the hyperparameter set from our section 1, and consider both model size scaling and data scaling. Identify new cases where the scaling law slopes vary, but still remain within the linear regime. (10 runs)
- (c) Finally, let's break the scaling law entirely. Find interventions (potentially radical ones) within our hyperparameter space which lead to strong deviations from the power law functional form. What types of intervention seem to really break scaling laws, and in what compute regimes and scaling axes does this happen? (20 runs)

We have now seen examples of all 3 cases, and so we should appreciate the fact that scaling laws do show up often, but that we shouldn't take them for granted. In our quiz, we will assess you on your ability to identify cases that we expect to scale, vs those which we would not. As with any scaling forecast, we will provide a preliminary pilot scaling plot, and ask you whether you trust the naive scaling projections.

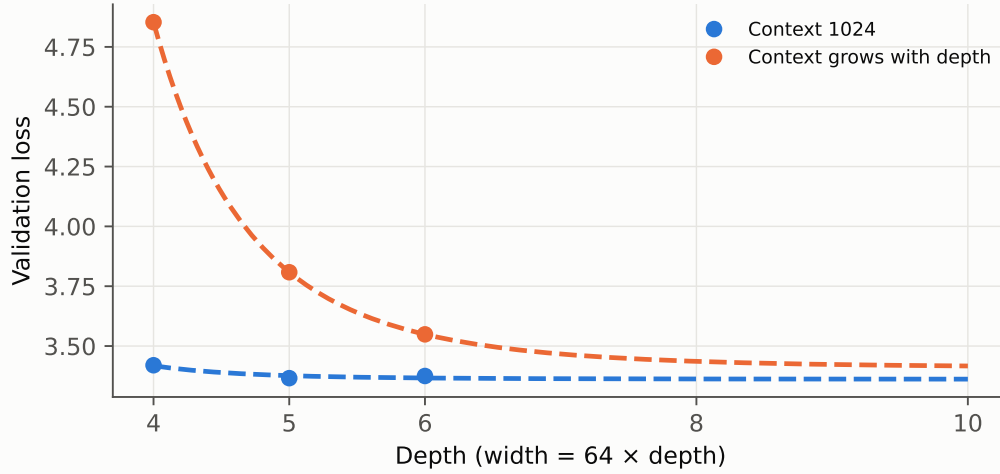
Example question

Difficulty: Hard

Note: The current version of the codebase does not easily support varying context length with scale, and so we do not provide a diff, nor do we want you to run this experiment yourself (even if you have the compute). Consider this question as an illustrative example of the types of generalization we want you to be able to do.

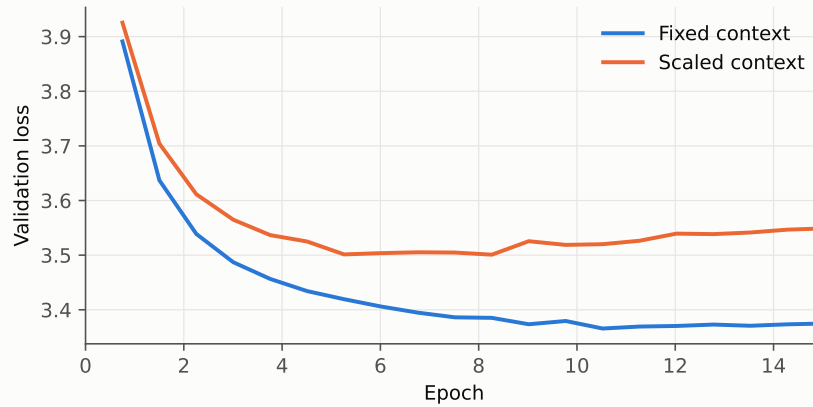
We compare a fixed-context recipe with one whose context grows with depth. Both use the same 39.3216M-token training data repeated 15 times, or 589.824M training tokens. Both use LR 0.003, linear decay, 1% warmup, and 61,440 tokens per optimizer step. Gradient accumulation is used to keep the global batch size fixed, and validation always uses the same 1024-token sequences. Pilot depths are d4/d5/d6; the held-out endpoint is d10.

The following measured pilots are fitted with $L = aC^{-\alpha} + \varepsilon$, where C is the $6ND$ body-compute proxy relative to d8 at the same token horizon:



Pilot runs at d4, d5, and d6. Dashed lines extrapolate the fitted curves to d10.

The fitted curves predict a d10 gap (scaled minus fixed context) of +0.0550 nats. As additional context, you can look at the validation loss curve for the d6 run.



Validation curves for the d6 pilots.

We now run both recipes at d10. Predict the validation-loss gap $\Delta = L_{\text{scaled}} - L_{\text{fixed}}$ in nats, and explain whether you trust the pilot extrapolation.

- ☐ $\Delta \leq -0.15$
- ☐ $-0.15 < \Delta \leq 0.03$
- ☐ $0.03 < \Delta \leq 0.15$
- ☐ $\Delta > 0.15$

Engineering scaling

Finally, we end this section with a bit of expository discussion on what a “scaling recipe” is. Thus far, we have assumed that there is already a family of models which differ in compute scale and have tested to see if they follow a scaling law. However, how should we pick these models along a scaling ladder? this is the problem of finding a scaling recipe.

As a warmup, let us start by writing down a very basic naive scaling recipe. Anchored at the standard d8 run ($C = 1$), it varies four quantities at once as a function of the compute multiplier C : tokens grow as $\sqrt{C}$, depth and width each grow as $C^{1/6}$ (preserving the ladder’s fixed aspect ratio, so that parameters $N \propto \text{depth} \cdot \text{width}^2$ grow as $\sqrt{C}$), and the peak learning rate shrinks as $C^{-1/6}$. This is a vanilla “Chinchilla”-style recipe which splits compute evenly between parameters and tokens while inverse-scaling the learning rate with width. Concretely:

Quantity	Scaling	$C = 1/64$	$C = 1$	$C = 64$
model	$\text{depth} \propto C^{1/6}$	d4	d8	d16
width	$64 \cdot \text{depth}$	256	512	1024
training tokens	$\propto \sqrt{C}$	76.8M	614.4M	4915.2M
peak learning rate	$\propto C^{-1/6}$	0.006	0.003	0.0015

You can, of course, come up with any recipe that you’d like with one important constraint: *your recipe must fulfil the FLOPs compute equation*

$$\text{FLOPs} \approx 6ND, \quad N \approx 14.5 \cdot \text{depth} \cdot \text{width}^2,$$

where D is the number of training tokens and N is the non-embedding parameter count (4 width^2 per layer for attention plus $3 \times 3.5 \text{width}^2$ for the gated MLP; at d8 this gives $N \approx 30.4\text{M}$, matching the parameter table). The factor of 6 counts the forward and backward matrix multiplies per parameter per token.

If you want a derivation of this equation, you should take a look at the CS336 assignments / lectures, which go over these ideas in detail. For any deep neural network, you can follow a similar process to derive the FLOPs cost as a function of your hyperparameters. Note that the true x-axis on any scaling law should probably be the true cost of training the model (often in dollars), but we will ignore that complexity for the purpose of our assignment. We will return to this idea of a scaling recipe repeatedly in the subsequent assignments, as we figure out the right scaling recipe for different optimizers and architectures.

Stochasticity

When we run experiments at the small scale, we often see only small improvements or changes from interventions we make. When are these significant, and when are they just noise? Run-to-run variation is a fact of life in machine learning, and some settings (like reinforcement learning) have it much worse than others. Whenever we start working in a domain, we should have a good sense of the order of magnitude difference in loss or accuracy where we go from the noise floor to a significant difference.

What are the sources of variability in a standard, deep neural network training run? There are three (maybe four, depending on how you count) sources of variability.

- Initialization - Essentially all deep neural networks use random initializations to do symmetry breaking. This process seeds some randomness in the network
- Data order and choice - Most networks are trained using stochastic gradients. The choice of which examples to sample at each step injects stochasticity.

- Hardware nondeterminism - GPUs do not guarantee deterministic execution due to non-associativity of floating point operations.
- (Bonus) Chaotic training trajectories - Unstable, chaos-like behavior in optimization trajectories amplifies even infinitesimal variation.

Chaotic training is not itself a source of randomness, but is a major contributor to randomness, as it amplifies any existing source of randomness (like hardware nondeterminism) that is small into a large, macroscopically observable source of randomness.

You should have an intuitive sense of how much each of these sources (and their sum total) affect the loss and training trajectory of the model. Understanding this is important in not only being able to tell when your methods lead to improvements, but also in identifying anomalous training phenomena that you don't expect to see just by chance.

As a warmup, let us start by measuring all the sources of variation that we have at hand,

Problem 3: Measuring variation (25 runs)

Take the d8 baseline, and add instrumentation to vary the first 3 stochasticity components above either in isolation or jointly. In the case of hardware nondeterminism, run your code on at least two different machines (H100/A100) to reliably induce nondeterminism effects. To test reproducibility on a fixed GPU, the following code enables PyTorch determinism and defines two identical reference runs.

```
experiments/a1_basics/p3_measuring_variation.py
RUNS = [
    TrainConfig(
        deterministic=True,
        run_name_suffix="deterministic-reference-1",
    ),
    TrainConfig(
        deterministic=True,
        run_name_suffix="deterministic-reference-2",
    ),
]
```

- First, vary all 3 natural sources of variation, and look at the distribution of terminal losses and variability of the train loss curve. What is the general order of magnitude of loss variation? What's the shape of the loss distribution? What factors change the training loss curve? (3 runs)
- Isolate the 3 sources of variation, and see what fraction of the full variation you end up with when you vary each one. What conclusion do you draw on the basis of your observations? (6-12 runs)
- Explore hyperparameter variations. Do any choices of hyperparameters significantly change the variability? For more capable (larger/longer training) models with lower losses, how does the run-to-run variation change? (10-16 runs)

These experiments should give you a sense of how much randomness you should expect. We will now explore the fourth "source of variation" of how randomness amplifies during training. Chaotic training dynamics is a deeper set of phenomena, which we will return to later in our optimizer module. However, for now, we will make some micro-perturbations, and see how they amplify over training.

Problem 4: Amplification of randomness (15 runs)

We want to trace how nearly infinitesimal randomness sources amplify during training. This starting comparison edits one token in the first shuffled sequence: after the training data is loaded and shuffled, token position 100 of row 0 is changed to token ID 17. The training code asserts that this token was not already 17.

```
experiments/a1_basics/p4_amplification.py
RUNS = [
    TrainConfig(deterministic=True),
    TrainConfig(deterministic=True, perturb_one_token=True),
]
```

- Explore changing a single data point (adding, deleting, etc) during the training process, while fixing *all* sources of randomness. How much variation do you get in terminal training loss? (1-5 runs)
- Continue to fix all standard sources of randomness. Track the loss variation over time and understand how the magnitude of the perturbation at time T, and the number of steps we take after T, both affect the run-to-run variation. (10-14 runs)

Now you should feel quite comfortable in judging whether a loss difference of 0.001 or 0.0001 is significant in our standard d8 setup. A quiz question might test a combination of our intuitions.

Example question**Difficulty: Medium**

Train the standard d8 recipe at paired model/data seeds 4 through 13. Independently repeat this ten-seed experiment with (A) global batch size 16 instead of 64 and (B) peak LR 0.009 instead of 0.003. Assume the same GPU is used for all 30 runs.

Which independent change makes the sample standard deviation of final validation loss **more than twice** that of the standard recipe?

- Only the smaller batch.
- Only the higher learning rate.
- Both changes.
- Neither change.

Each sample SD is computed across the ten final losses using the formula $\sqrt{\sum_{i=1}^n (x_i - \mu)^2 / (n - 1)}$.

```
--- a/experiments/a1_basics/p3_measuring_variation.py
+++ b/experiments/a1_basics/p3_measuring_variation.py
@@
+RUNS = [
+    TrainConfig(model_seed=seed, data_seed=seed, **changes)
+    for label, changes in [
+        ("standard", {}),
+        ("batch16", {"batch_size": 16}),
+        ("lr009", {"learning_rate": 0.009}),
+    ]
+    for seed in range(4, 14)
+]
```

Explain your reasoning in your writeup.

Debugging and fixing your model

One of the challenges of modern machine learning is that it is difficult to know why models are not working, and fixing models is a dark art. Models can suddenly diverge during training, or losses can plateau after a long period of optimization. When these things happen, we must have ways of looking inside the model to get a better picture of what is happening.

Diagnosing problems

What tools do we have to get a better picture of what's happening in a neural network?

- Loss curves: the shape of the loss curves can actually tell us quite a bit about learning rates, model behaviors, and so on.
- Gradient statistics: gradient magnitude is one direct way to see how training becomes unstable or stall.
- Activations: anomalous behaviors in activations could identify failures in our architectures or scaling recipes.

A complete understanding of how to debug a model is outside the scope of this class (and in fact, is not a solved problem even for experts). However, the first step to diagnosis is being able to know what is normal. To this end, we will track many common statistics about training in an effort to understand what “normal training” looks like in a model.

Problem 5: Loss curve augury (5 runs)

One of the most basic and useful diagnostic items is the training loss curve. The exact shape of the loss curve, both at a macroscopic level (how it goes up and down smoothly) and microscopic level (the local fluctuations in loss as well as spikes) can tell us quite a bit about the health of a training run.

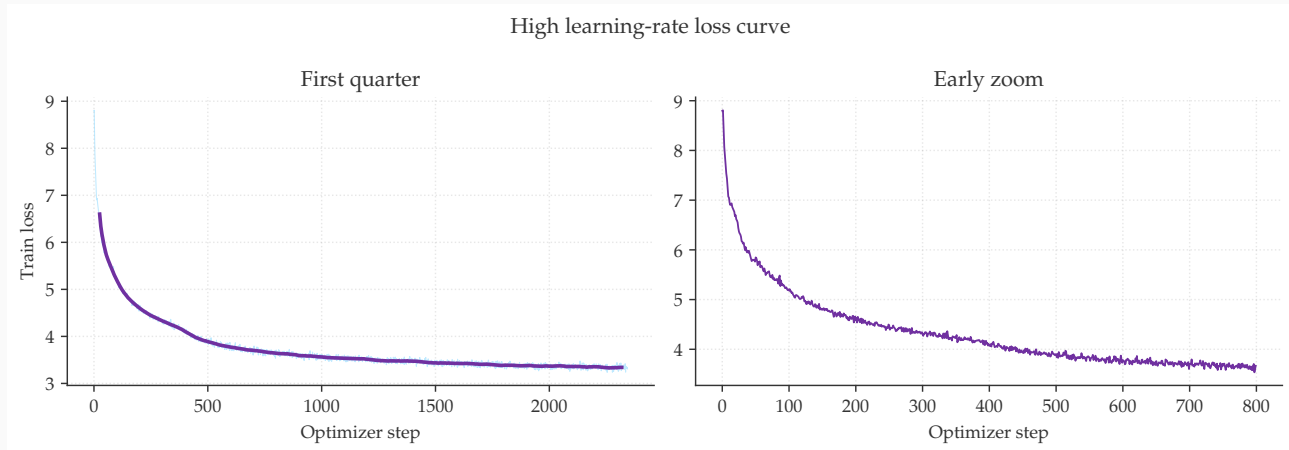
- (a) Look at your hyperparameter optimization runs that vary the basic optimizer hyperparameter components (learning rate, momentum via `beta1`, batch size, learning rate schedule) and get a sense of what factors change the macro loss-curve shapes and how. Do learning curves generally follow some predictable shapes? (0-5 runs)
- (b) Now that we understand the macro structure of loss curves, let's explore the microstructure. What hyperparameters increase or decrease the “smoothness” of the loss curve as it descends? you can start with the obvious one of batch size, but is this the only one? (0 runs)
- (c) Look back at all your old runs in weights and biases. Are there any runs from the past that significantly changed the loss curve shapes (either macro or micro)? Try to build a good mental model of what factors strongly affect these shapes, and what good vs bad training looks like. (0 runs)

An associated quiz question to this task might look like the following.

Example question

Difficulty: Easy

You are a training run babysitter, monitoring your friend's d8 training run loss curve. After a short while, you see the following loss curve



First 25% of the run; left: raw losses and a 51-step mean; right: an unsmoothed zoom.

- (a) Based on this loss curve, make a guess about what the run configuration for this training run was

Here are 4 options, with their associated diffs from d8 defining their configurations.

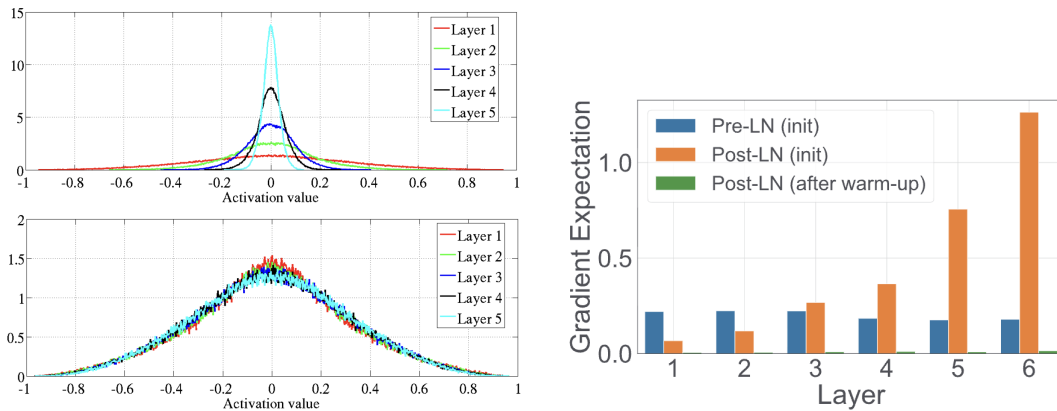
The patch defines all four candidate runs.

```
--- a/experiments/a1_basics/p5_loss_curve_augury.py
+++ b/experiments/a1_basics/p5_loss_curve_augury.py
@@
-RUNS = [
-    TrainConfig(),
-]
+RUNS = [
+    TrainConfig(),
+    TrainConfig(batch_size=256),
+    TrainConfig(learning_rate=0.0003),
+    TrainConfig(learning_rate=0.009),
+]
```

- ☐ A Standard d8 (no diff)
- ☐ B Batch size 256
- ☐ C Learning rate 0.0003
- ☐ D Learning rate 0.009
- (b) Now that you have made a guess about the run configuration, try to guess the terminal loss to see if this will actually be a good training run, relative to the standard d8 loss Use validation-loss gap relative to the standard run (2.926):
- ☐ a At most +0.05
- ☐ b More than +0.05, at most +0.5
- ☐ c More than +0.5, but a finite final loss
- ☐ d Numerical divergence (non-finite loss)

Beyond loss curves, we can look at more advanced statistics such as gradient statistics and activations that we can inspect. Now that we have an understanding of basic levers that change the training dynamics of models, we can inspect them in conjunction with parameters, activation, and gradient norms.

Careful inspection and reasoning of these statistics have been foundational to deep learning. Activation tracking was instrumental to some of the earliest deep learning discoveries, such as the Xavier initialization, while reasoning about gradient norm magnitudes were key in developing pre-norm layer norm placements in the transformer.



Xavier initialization and Pre-norm were both developed by thinking about internal statistics.

More generally, the activations give us much more fine-grained pictures of what happens in the forward pass, the gradients give us a good picture of the backward pass, and the parameter statistics can tell us about invariances and training dynamics. All of these can give us much better views into model behavior than looking at loss curves alone.

Problem 6: Activations and gradient norms (0-5 runs)

Let us now track the activations across different transformer components and gradient norms. Unlike the case of the loss, this is a much more granular and high-dimensional object and you have significantly more freedom to explore the right types of statistics to track and measure. We will start you off with some basic suggestions, and then have you explore the richer space.

- Take the start, end, and middle of the training trajectory of a standard d8 run. Visualize statistics of the parameter, activation, and gradient norm at that point. You will want to look at the RMS norm (logged by default) but for any runs you may do, you may want to look at other statistics like the max or quantiles. Do so at the start and end of various transformer operations (MLP, Attention) across different layers. Build a mental model of how activations and gradients evolve across training trajectory and depth. (0 runs - these statistics are logged in W&B by default)
- Go through your past experiments, and look at your W&B logs. You should be able to look at existing training statistics across different layers. How do your various interventions change these statistics? (0 runs).
- We want you to now be able to manipulate these gradient and activation values. Pick a few core statistics from the first part (like the l2 norm of the gradient/activations) and figure out how to manipulate these using hyperparameters. Can you make the parameter, gradient or activation statistics larger or smaller uniformly? what about just at the start or end of training? (0-5 runs)

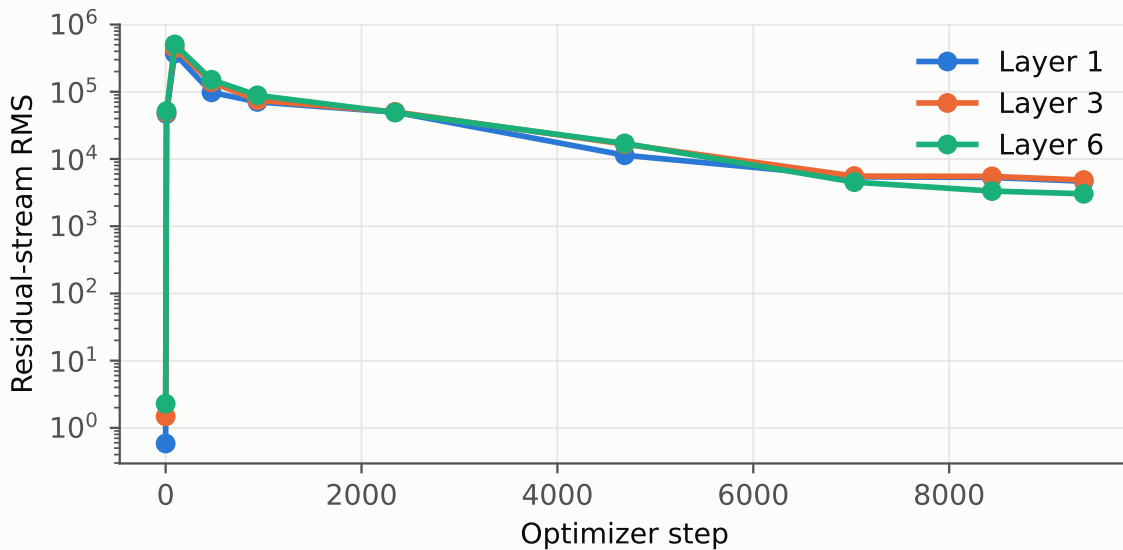
These metrics allow us to identify anomalous training situations. In this assignment and quiz, we have not yet introduced the architecture and optimizers variations that break models in complex and subtle ways. However, we can still quiz you on some basic ways to break models that you need to identify via diagnostics

Example question

Difficulty: Easy

You are given the following traces of activation statistics at layers 1, 3, and 6.

Layers are numbered from 1; the statistic is RMS of the post-block residual stream on the same eight validation sequences, sampled at fixed training steps.



You need to figure out if this training run is from a healthy run (e.g. a standard d8 run) or a completely crazy set of hyperparameter settings, given by this diff

This intervention is not a TrainConfig-only change. It is implemented by editing the experiment file to add a custom metric logger that registers temporary forward hooks on selected layers.

```
--- a/experiments/a1_basics/p6_activation_gradient_norms.py
+++ b/experiments/a1_basics/p6_activation_gradient_norms.py
@@
+from metric_logging import AFTER_EVAL
+import torch
+
+def log_activation_rms(ctx):
+    input_ids = next(iter(ctx.val_batches["val"]))[:8]
+    input_ids = input_ids.to(next(ctx.model.parameters()).device)
+    stats, handles = {}, []
+    for layer_idx in [0, 2, 5]:
+        def hook(module, inputs, output, layer_idx=layer_idx):
+            stats[f"layer_{layer_idx + 1}_residual_rms"] = (
+                output.detach().float().square().mean().sqrt()
+            )
+        handles.append(ctx.model.model.layers[layer_idx].register_forward_hook(hook))
+    was_training = ctx.model.training
+    ctx.model.eval()
+    try:
```

```
+         with torch.no_grad():
+             ctx.model(input_ids=input_ids)
+         finally:
+             for handle in handles:
+                 handle.remove()
+             if was_training:
+                 ctx.model.train()
+         return stats
+
+# Add MetricLogger(event=AFTER_EVAL, fn=log_activation_rms) to each run.
```

Choose one of the two options - d8 or alternative.

☐ Standard d8 ☐ Alternative

Optional exploration

To explore the topics in this assignment more broadly, construct your own prediction problem. This final exercise is not related to a quiz question, but can lead to bonus points on the quiz.

Problem 7: Your own prediction problem (5-10 runs)

You have now explored the phenomena in this assignment thoroughly, and may have found unexpected or surprising phenomena. We would like to help make better, more interesting quizzes based on your insights.

Include the following in your exploration:

- (a) A question writeup, similar to the sample questions in this assignment. Answers should be objectively checkable based on the execution of a piece of code (ideally, multiple choice or numerical prediction).
- (b) A code diff and executable code which resolves to an answer in the question (e.g. prints the loss). The code diff should be compact (<100 lines of code)

Aim for a fair, novel question: its outcome should be possible to reason about using the assignment, while testing an insight beyond the supplied examples. Course staff may use interesting questions in a future quiz.